

Research Article Helper Report

Fire and Smoke Detection with False Alarm Reduction, Fire Progression Prediction, and Automated Incident Reporting

Status: Training in progress. Sections marked [ADD AFTER TRAINING] must be filled once YOLOv8m and Temporal Transformer training completes.

SUGGESTED PAPER TITLE OPTIONS

- "Real-Time Fire and Smoke Detection with Three-Stage False Alarm Reduction and Temporal Transformer-Based Progression Prediction"
- "A Novel Multi-Stage Fire Detection Framework with AI-Driven Incident Report Generation"
- "Beyond Detection: Predicting Fire Progression Using Temporal Transformers with Automated Natural Language Incident Reporting"

Recommended: Option 1 — most descriptive, covers all three novelties clearly.

ABSTRACT (DRAFT)

Fire detection systems based on deep learning have demonstrated high accuracy in identifying fire and smoke in visual data. However, existing approaches suffer from high false alarm rates caused by visually similar stimuli such as sunsets, reflections, and illumination changes, and they provide no information about how a detected fire will evolve over time. This paper presents a comprehensive real-time fire and smoke detection framework that addresses both limitations. The proposed system combines a YOLOv8m object detector with a novel Three-Stage False Alarm Reduction Framework consisting of HSV color space verification, temporal persistence filtering, and Laplacian texture chaos analysis. A Temporal Transformer model is trained on sequential fire detection features to predict fire progression — classifying future behavior as stable, growing, or critical — along with quantitative fire area estimates at 5-second and 10-second horizons. Finally, an automated Natural Language Generation module powered by a locally deployed large language model converts structured detection metadata into human-readable incident descriptions and safety recommendations in real time. Experiments on the D-Fire dataset demonstrate [ADD AFTER TRAINING]. The complete system runs at [X] FPS on an NVIDIA RTX 4050 GPU, making it suitable for real-time deployment.

Keywords: Fire detection, smoke detection, YOLOv8, false alarm reduction, Temporal Transformer, fire progression prediction, natural language generation, incident reporting, real-time surveillance.

1. INTRODUCTION

1.1 Problem Statement

Fire is one of the most destructive natural and man-made disasters, causing significant loss of life, property, and environment annually. Early and accurate detection is critical to minimizing damage. Traditional fire detection systems rely on heat or gas sensors, which require physical proximity to the fire source. Camera-based fire detection using deep learning offers a non-contact, wide-area monitoring solution.

However, current camera-based detection systems face two key limitations:

1. **High false alarm rate** — Objects with similar visual characteristics to fire and smoke (sunsets, red indicator lights, vehicle tail lights, construction dust) trigger false detections, reducing system reliability and operator trust.
2. **Detection without prediction** — Current systems only answer "Is fire present?" not "How will the fire develop?" This limits the time available for evacuation and emergency response.

1.2 Contributions

This paper makes the following contributions:

1. **Three-Stage False Alarm Reduction Framework** — A novel post-detection verification pipeline combining HSV color space analysis, temporal persistence filtering, and Laplacian texture chaos analysis, operating sequentially to reject false positives before alert generation.
 2. **Temporal Transformer for Fire Progression Prediction** — A transformer-based sequence model that processes 15 consecutive frames of structured detection features to predict whether fire will remain stable, grow, or become critical within the next 5–10 seconds, along with quantitative fire area forecasts.
 3. **Automated LLM-Powered Incident Reporting** — A natural language generation module that uses a locally deployed large language model (Ollama/Phi-3 Mini) to convert structured detection data into human-readable incident descriptions and safety recommendations, with rule-based fallback ensuring reliability under all conditions.
 4. **End-to-End Real-Time Pipeline** — All three components are integrated into a single pipeline that processes live video frames and produces annotated output with detection overlays, prediction labels, risk scores, and incident reports simultaneously.
-

2. RELATED WORK

2.1 Image-Based Fire Detection

Early deep learning approaches for fire detection adapted CNNs for binary classification (fire vs. no-fire). Sharma et al. (2017) demonstrated that pre-trained VGG-16 features could effectively distinguish fire from non-fire images. Subsequently, YOLO-based detectors were applied for localizing fire regions. Cao et al. (2019) proposed a modified YOLOv3 for fire and smoke detection. Muhammad et al. (2018) introduced an efficient deep learning approach for early fire detection in surveillance systems.

The YOLOv8 family, introduced by Ultralytics (2023), achieves state-of-the-art detection accuracy with improved anchor-free architecture and C2f modules, making it particularly suitable for real-time applications.

Gap: Most existing detection approaches do not perform post-detection verification, leading to false alarms from visually similar stimuli.

2.2 False Alarm Reduction

Several works have addressed false alarm reduction in fire detection. Chenebert et al. (2011) used color and texture features for fire pixel classification. Ko et al. (2012) proposed motion analysis to distinguish real fire from fire-like objects. Celik & Demirel (2009) developed HSV-based fire color models. However, these methods operate at the pixel level and are not designed to work as post-hoc verification stages on top of deep learning detectors.

Gap: No existing work combines HSV verification, temporal persistence, and texture chaos analysis as a three-stage post-detection filter on top of a YOLOv8 detector.

2.3 Temporal Modeling for Fire

A few works have explored temporal information for fire detection. Foggia et al. (2015) used background subtraction and temporal features. Steffens et al. (2017) analyzed temporal patterns in fire pixel sequences. However, these

approaches focus on improving detection rather than predicting future fire behavior.

Transformer architectures (Vaswani et al., 2017) have been applied to video understanding tasks. Vision Transformers and Video Swin Transformers have shown strong performance on action recognition and temporal modeling. However, applying lightweight transformers to structured fire feature sequences for progression prediction has not been explored.

Gap: No existing work predicts fire progression (stable/growing/critical) using a Temporal Transformer on structured detection feature sequences.

2.4 Natural Language Generation in Safety Systems

NLG has been applied in weather forecasting (Reiter et al., 2005), medical reporting, and traffic incident reporting. The emergence of lightweight local LLMs (Phi-3, Llama 3.2) makes real-time, offline LLM-powered reporting feasible. No existing fire detection system integrates LLM-based natural language incident report generation.

Gap: No fire detection system generates human-readable incident reports using a local LLM in real time.

2.5 Papers to Cite in References

- Vaswani et al. (2017) — "Attention Is All You Need" — Transformer architecture
- Redmon & Farhadi (2018) — YOLOv3 — YOLO background
- Jocher et al. (2023) — Ultralytics YOLOv8 — base detector
- Muhammad et al. (2018) — "Efficient Deep CNN-Based Fire Detection"
- Celik & Demirel (2009) — "Fire detection in video sequences using a generic color model"
- Foggia et al. (2015) — "Real-time fire detection for video-surveillance applications"
- Microsoft (2024) — Phi-3 Technical Report — local LLM

3. SYSTEM ARCHITECTURE

3.1 Overview

The proposed system consists of four sequential modules:

```
INPUT VIDEO FRAME
|
v
+-----+
| Module 1: YOLOv8m | <- Object detection (fire, smoke)
| Detector |
+-----+
| raw detections
v
+-----+
| Module 2: Three-Stage | <- False alarm rejection
| False Alarm Reduction | Stage 1: HSV Color
| | Stage 2: Temporal Persistence
| | Stage 3: Texture Chaos
+-----+
| verified detections
v
+-----+
| Module 3: Temporal | <- Fire progression prediction
| Transformer Predictor | stable / growing / critical
| | + risk score + area forecast
+-----+
| prediction
```

```
v
+-----+
| Module 4: NLG Incident | <- Natural language report
| Reporter (Ollama LLM) | description + recommendation
+-----+
|
v
ANNOTATED FRAME + REPORT
```

4. METHODOLOGY

4.1 Module 1 — YOLOv8m Fire and Smoke Detector

Architecture

Parameter	Value
Architecture	YOLOv8m
depth_multiple	0.67
width_multiple	0.75
Total parameters	~25.9 million
Input resolution	640 x 640
Detection classes	2 (smoke=0, fire=1)
Anchor type	Anchor-free
Backbone	CSP-Darknet with C2f modules
Neck	PAN-FPN
Head	Decoupled detection head

Training Dataset — D-Fire

Split	Images
Train	[ADD AFTER TRAINING]
Validation	[ADD AFTER TRAINING]
Source	kaggle: sayedgamal99/smoke-fire-detection-yolo

Training Configuration

Hyperparameter	Value
Epochs	50 (early stopping patience=15)
Batch size	8 (6GB VRAM)
Image size	640 x 640
Optimizer	SGD with momentum

Hyperparameter	Value
Learning rate (lr0)	0.01
Momentum	0.937
Weight decay	0.0005
Warmup epochs	3
Mixed precision (AMP)	Enabled
Mosaic augmentation	1.0
Pretrained	COCO
Hardware	NVIDIA RTX 4050 (6GB VRAM)

Detection Results — [ADD AFTER TRAINING]

Metric	Overall	Fire	Smoke
mAP@0.5	[ADD]	[ADD]	[ADD]
mAP@0.5:0.95	[ADD]	[ADD]	[ADD]
Precision	[ADD]	[ADD]	[ADD]
Recall	[ADD]	[ADD]	[ADD]
Inference speed	[ADD] ms/frame	-	-
Training time	[ADD] hours	-	-

4.2 Module 2 — Three-Stage False Alarm Reduction Framework

Stage 1 — HSV Color Space Verification

Each detected bounding box is cropped and converted to HSV color space. A color mask is applied based on fire/smoke spectral characteristics.

Fire color model:

```
mask1 = H in [0, 35] AND S > 100 AND V > 100 (red-orange-yellow)
mask2 = H in [160, 180] AND S > 100 AND V > 100 (red wraparound)
ratio = count(mask1 OR mask2) / total_pixels
accept if ratio > 0.12
```

Smoke color model:

```
mask = S in [0, 60] AND V in [80, 220] (low-saturation grey/white)
ratio = count(mask) / total_pixels
accept if ratio > 0.10
```

Rejects: sunsets, tail lights, orange barriers.

Stage 2 — Temporal Persistence Filter

A detection at frame t is accepted only if a spatially overlapping detection (IoU > 0.10) existed in at least one of the previous W=3 frames:

```
IoU(box_t, box_{t-k}) > 0.10 for some k in {1, 2, 3}

IoU = intersection_area / (area_A + area_B - intersection_area)
```

Grace period applied for first W-1 frames of a new stream.

Rejects: camera flashes, single-frame sensor noise, sudden lighting spikes.

Stage 3 — Laplacian Texture Chaos Analysis

Fire and smoke have chaotic, high-frequency textures. Laplacian variance measures texture complexity:

```
L(x,y) = d2I/dx2 + d2I/dy2
texture_score = Var(L(I))

Fire threshold: texture_score > 200.0
Smoke threshold: texture_score > 80.0
```

Rejects: uniform red walls, indicator lights, smooth reflective surfaces.

False Alarm Reduction Results — [ADD AFTER TRAINING]

Stage	Detections Rejected	False Alarm Reduction
Stage 1 (HSV)	[ADD]	[ADD]%
Stage 2 (Persistence)	[ADD]	[ADD]%
Stage 3 (Texture)	[ADD]	[ADD]%
All Three Combined	[ADD]	[ADD]%
True Positive Retention	[ADD]	[ADD]%

4.3 Module 3 — Temporal Transformer for Fire Progression Prediction

Problem Formulation

Given T=15 consecutive frames of fire features $X = \{x_1, ..., x_{15}\}$ where x_t in R^8 , predict:

- y_g in {stable, growing, critical} — growth classification
- y_r in [0, 100] — risk score
- y_{a5} in R^+ — predicted fire area in 5 seconds
- y_{a10} in R^+ — predicted fire area in 10 seconds

Feature Vector (8-dimensional, per frame)

Index	Feature	Description
0	fire_area	Fire bbox area / frame area
1	smoke_area	Smoke bbox area / frame area
2	fire_count	Number of fire detections
3	smoke_count	Number of smoke detections
4	avg_confidence	Mean detection confidence
5	growth_rate	$fire_area_t - fire_area_{t-1}$

Index	Feature	Description
6	centroid_x	Normalized fire centroid X
7	centroid_y	Normalized fire centroid Y

Model Architecture

```
Input: (B x 15 x 8)
|
v
Linear(8 -> 64)
|
v
Sinusoidal Positional Encoding (d_model=64)
|
v
TransformerEncoder:
layers=3, heads=4, d_model=64, ff=256, dropout=0.1
|
v
LayerNorm(64) -> Mean pooling over T=15 -> R^64
|
+-----+-----+-----+
| | | |
v v v v
growth_head risk_head area_5s_head area_10s_head
Linear(64,32) Linear(64,32) Linear(64,32) Linear(64,32)
ReLU ReLU ReLU ReLU
Linear(32,3) Linear(32,1) Linear(32,1) Linear(32,1)
Sigmoid x100 ReLU ReLU
| | | |
v v v v
logits(3) risk in[0,100] area_5s area_10s

Total parameters: ~85,000
```

Self-Supervised Labeling Strategy

Labels are automatically derived from YOLO detections using fire area growth rate — no manual annotation required:

```
growth_5s = (fire_area_{t + fps*5} - fire_area_t) / fire_area_t

growth_5s > 0.60 -> critical (label=2)
growth_5s > 0.30 -> growing (label=1)
otherwise -> stable (label=0)

risk_score = min(100, |growth_5s| x 100)
```

Training Dataset — UniDataPro Fire Videos

Property	Value
Source	kaggle: unidpro/fire-and-smoke-dataset
Videos	85 fire/smoke videos
Sequence length	15 frames (sliding window)
Augmentation	Gaussian noise x5 copies (std=0.01)
Train/Val split	80% / 20%

[ADD AFTER TRAINING]

Split	Sequences	Stable	Growing	Critical
Train	[ADD]	[ADD]	[ADD]	[ADD]
Validation	[ADD]	[ADD]	[ADD]	[ADD]

Training Configuration

Hyperparameter	Value
Optimizer	Adam
Learning rate	0.001
Weight decay	1e-4
Batch size	64
Max epochs	50
Early stopping patience	10
LR scheduler	ReduceLROnPlateau (factor=0.5)
Gradient clipping	max_norm=1.0

Combined multi-task loss:

$$L = L_CE(growth) + 0.3 * L_MSE(risk) + 0.2 * L_MSE(area_5s) + 0.2 * L_MSE(area_10s)$$

Temporal Transformer Results — [ADD AFTER TRAINING]

Metric	Value
Validation loss	[ADD]
Growth accuracy (overall)	[ADD]%
Accuracy — stable	[ADD]%
Accuracy — growing	[ADD]%
Accuracy — critical	[ADD]%
Risk score MAE	[ADD]
Area-5s MAE	[ADD]
Area-10s MAE	[ADD]
Inference time	[ADD] ms

4.4 Module 4 — Automated NLG Incident Reporter

Spatial Region Mapping

Frame divided into 3x3 grid for location naming:

```

+-----+-----+-----+
| northwest | northern | northeast |
+-----+-----+-----+
| western | central | eastern |
+-----+-----+-----+
| southwest | southern | southeast |
+-----+-----+-----+

region_col = int(centroid_x / width * 3)
region_row = int(centroid_y / height * 3)

```

Trend Analysis (60-frame history)

```

slope = (area[-1] - area[-10]) / 10

slope > 0.005 -> "increasing"
slope < -0.005 -> "decreasing"
otherwise -> "stable"

```

LLM Prompt Template

You are a fire safety monitoring AI. Write exactly 2 sentences based on the data below.
Sentence 1: describe the incident (what, where, how long, severity).
Sentence 2: give a safety recommendation.

- Fire detections: {fire_count}
- Smoke detections: {smoke_count}
- Confidence: {avg_conf}%
- Location: {region}
- Trend: {trend}
- Consecutive frames: {consecutive}
- Severity: {severity}
- AI fire progression: {growth_label} (risk: {risk}/100)

Non-Blocking Design

```

Frame N (interval fires)
-> Previous report stays on screen
-> Background thread starts
-> Ollama called (timeout 10s)
Success -> LLM report saved
Failure -> Rule-based report generated in thread
-> ready_report set

Frame N+k
-> ready_report swapped to display
-> No flicker, Ollama always tried first

```

Example LLM Output

Description:

"High-confidence fire and smoke detected in the northeast region of the monitored area, persisting across 18 consecutive frames with increasing smoke density indicating active fire spread."

Recommendation:

"Immediate evacuation of the northeast zone is recommended; contact emergency services and activate fire suppression systems without delay."

5. IMPLEMENTATION

Hardware

Component	Specification
GPU	NVIDIA GeForce RTX 4050 Laptop GPU
VRAM	6 GB GDDR6
CUDA	13.3
OS	Windows 11
CPU	[ADD your CPU]
RAM	[ADD your RAM]

Software Stack

Package	Version	Purpose
Python	3.10.11	Runtime
PyTorch	2.6.0+cu124	Deep learning
Ultralytics	8.x	YOLOv8m
OpenCV	4.x	Frame processing
NumPy	1.26.4	Arrays
Ollama	Latest	Local LLM server
Phi-3 Mini	2.2B params	Incident report LLM

6. EXPERIMENTS & ABLATION STUDY

Experiment A — Detector Comparison

Configuration	mAP@0.5	FPS
YOLOv8n (baseline)	[ADD]	[ADD]
YOLOv8m (proposed)	[ADD]	[ADD]

Experiment B — False Alarm Reduction Stages

Configuration	False Alarms	True Positives Retained
No verification (baseline)	[ADD]	100%
Stage 1 only (HSV)	[ADD]	[ADD]%
Stage 1+2 (HSV+Persistence)	[ADD]	[ADD]%
All 3 stages (proposed)	[ADD]	[ADD]%

Experiment C — Temporal Transformer

Configuration	Growth Accuracy	Risk MAE
No prediction (baseline)	—	—
With Temporal Transformer	[ADD]	[ADD]

End-to-End System Performance — [ADD AFTER TRAINING]

Metric	Value
FPS (all modules, GPU)	[ADD]
Total latency per frame	[ADD] ms
Memory usage (GPU)	[ADD] MB

7. COMPLETE RESULTS SUMMARY — [ADD AFTER TRAINING]

Metric	Value
YOLOv8m mAP@0.5	[ADD]%
YOLOv8m mAP@0.5:0.95	[ADD]%
YOLOv8m Precision	[ADD]%
YOLOv8m Recall	[ADD]%
False alarm reduction	[ADD]%
True positive retention	[ADD]%
Growth classification accuracy	[ADD]%
Risk score MAE	[ADD]
End-to-end FPS	[ADD]

8. PAPER WRITING CHECKLIST

Sections

- [] Abstract — update with real numbers after training
- [] Introduction — adapt Section 1 to paper format
- [] Related Work — expand with more citations from Section 2
- [] System Architecture — add block diagram figure
- [] Methodology — core of paper, use Section 4
- [] Implementation — hardware/software, Section 5
- [] Experiments & Results — fill all tables after training
- [] Discussion — limitations + future work
- [] Conclusion
- [] References

Figures to Create

- [] Figure 1: Full system architecture block diagram
- [] Figure 2: Three-stage false alarm reduction pipeline
- [] Figure 3: Temporal Transformer architecture
- [] Figure 4: Incident reporter flow (Ollama + fallback)
- [] Figure 5: Sample detections (true positives)
- [] Figure 6: False alarm rejection examples (one per stage)
- [] Figure 7: YOLOv8m training curves (loss, mAP vs epoch)
- [] Figure 8: Temporal Transformer training loss curve
- [] Figure 9: Incident report overlay screenshot

Numbers to Record After Training

- [] mAP@0.5, mAP@0.5:0.95, precision, recall (overall + per class)
- [] Best epoch (from early stopping)
- [] YOLOv8m training time
- [] Number of sequences extracted from 85 videos
- [] Class distribution (stable/growing/critical counts and percentages)
- [] Temporal Transformer: val loss, accuracy, MAE values
- [] Temporal Transformer training time
- [] End-to-end FPS with all modules running
- [] False alarms rejected per stage (run on held-out test set)

9. SUGGESTED JOURNALS / CONFERENCES

Venue	Type	Fit
Expert Systems with Applications (Elsevier)	Journal	Best fit — applied AI
IEEE Access	Journal	Good fit — broad engineering
Fire Safety Journal (Elsevier)	Journal	Domain-specific, strong fit
Engineering Applications of AI (Elsevier)	Journal	Good fit
Pattern Recognition Letters	Journal	Computer vision focus
IEEE CVPR / ICCV	Conference	High impact, competitive

Recommended first target: Expert Systems with Applications or IEEE Access — both accept applied AI papers, have reasonable review timelines, and are well matched to this work.